Kepler Exoplanet Detection

A machine learning pipeline for detecting exoplanets from Kepler mission light curve data, combining a hierarchical KNN+DTW classifier with a 2D Convolutional Neural Network. Built as part of MATH336 Mathematical Modelling at FLAME University.

Authors: Piya Shah, Saharsh Kanchan, Prithvi Dhyani

Overview

This project uses the **transit method** — detecting periodic dips in stellar brightness caused by an orbiting planet crossing the line of sight — to classify stars as exoplanet hosts or non-hosts from Kepler light curve data.

Two independent detection approaches were implemented and evaluated:

Approach	Accuracy	Exoplanet F1	Notes
Hierarchical KNN + DTW	95%	0.78	+10pp recall over baseline
2D CNN (phase-folded images)	**99.2%**	—	AUC: 0.97

Mathematical Background

The Transit Method

When a planet of radius R_p orbits a star of radius R_s, the fractional dip in observed stellar flux during transit is:

$$\frac{\Delta F}{F} = \left(\frac{R_p}{R_s}\right)^2$$

The duration of the transit T depends on the orbital period P, stellar radius R_s, and semi-major axis a:

$$T \approx \frac{P \cdot R_s}{\pi \cdot a}$$

Kepler measures stellar brightness continuously. A transit event appears as a periodic, symmetric dip in the light curve $F(t)$.

Pipeline

1. Data Engineering: Raw Pixels to Clean Flux

Raw Kepler data arrives as Target Pixel Files (TPFs) — 2D arrays of pixel intensities over time. Let $I(x, y, t)$ denote the intensity (electrons/s) at pixel (x, y) and time t.

Background Subtraction

Background signal $B(x, y, t)$ is estimated from pixels not containing the target star and subtracted:

$$I_{\text{cal}}(x, y, t) = I(x, y, t) - B(x, y, t)$$

Flat-Field Correction

Pixel-level sensitivity variations are corrected using a flat-field factor $F(x, y)$:

$$I_{\text{flat}}(x, y, t) = \frac{I_{\text{cal}}(x, y, t)}{F(x, y)}$$

Aperture Photometry

Flux is extracted by summing calibrated intensities over an optimal aperture $\mathcal{A}^*$:

$$F_{\text{raw}}(t) = \sum_{(x,y) \in \mathcal{A}} I_{\text{flat}}(x, y, t)$$

The optimal aperture maximises the signal-to-noise ratio:

$$\mathcal{A}^* = \arg\max_{\mathcal{A}} \frac{\sum_{(x,y)\in\mathcal{A}} I_{\text{flat}}(x, y, t)}{\sum_{(x,y)\in\mathcal{A}} \sigma^2(x, y, t)}$$

where $\sigma^2(x, y, t)$ is the per-pixel noise variance (photon + read noise).

Noise Modelling and Detrending

The observed flux contains both systematic and random noise:

$$F_{\text{obs}}(t) = F_{\text{true}}(t) + N_{\text{sys}}(t) + N_{\text{rand}}(t)$$

Systematic noise is modelled as a linear combination of regressors via a design matrix $\mathbf{X}(t)$:

$$N_{\text{sys}}(t) = \mathbf{X}(t) \cdot \boldsymbol{\beta}$$

PCA is applied to reduce the dimensionality of $\mathbf{X}$:

$$\mathbf{X} = \mathbf{U} \cdot \mathbf{\Sigma} \cdot \mathbf{V}^T$$

The corrected flux is then:

$$F_{\text{corr}}(t) = F_{\text{obs}}(t) - \mathbf{X}(t) \cdot \boldsymbol{\beta}$$

Uncertainty is propagated as:

$$\sigma_F^2 = \sigma_{\text{obs}}^2 + \sum_{i=1}^{k} \left(\frac{\partial F_{\text{corr}}}{\partial X_i} \cdot \sigma_{X_i} \right)^2$$

Pixel Response Function

The observed flux is shaped not just by astrophysical signals but by the instrument's Pixel Response Function (PRF):

$$F(t) = \iint I(x, y, t) \cdot \text{PRF}(x, y)\, dx\, dy$$

The PRF encodes the optical and electronic response of the detector, including the Point Spread Function (PSF). Residual PRF systematics are implicitly absorbed by the classifiers during training.

---

2. Distance-Based Classifier: Hierarchical KNN + DTW

Motivation

Comparing two light curves $X = (x_1, \ldots, x_N)$ and $Y = (y_1, \ldots, y_M)$ using Euclidean distance assumes perfect temporal alignment. In practice, transit timing variations, stellar activity, and instrument systematics cause phase shifts that make naive Euclidean comparison unreliable.

Dynamic Time Warping

DTW finds the optimal alignment between two time series by solving a dynamic programming problem.

Cost matrix — pairwise squared Euclidean distances:

$$D(i, j) = (x_i - y_j)^2$$

Cumulative cost matrix — computed via the recurrence:

$$C(i,j) = D(i,j) + \min \begin{cases} C(i-1,j) & \text{(insertion)} \\ C(i,j-1) & \text{(deletion)} \\ C(i-1,j-1) & \text{(match)} \end{cases}$$

with boundary conditions $C(0,0) = D(0,0), C(i,0) = \sum_{k=0}^{i} D(k,0), C(0,j) = \sum_{k=0}^{j} D(0,k)$.

DTW distance:

$$\text{DTW}(X,Y) = \sqrt{C(N,M)}$$

This allows flexible many-to-one alignments, making it robust to timing distortions that would mislead Euclidean distance. DTW can be viewed as a discrete subordinated process: the warping path π acts as the subordinator, mapping time indices of one series onto another via a non-decreasing sequence.

Computational Problem

Naively, DTW between two series of length m and n costs $\mathcal{O}(m \cdot n)$. Over a dataset with p training and q test examples this becomes:

$$\mathcal{O}(m \cdot n \cdot p \cdot q)$$

This is computationally infeasible for large datasets and long light curves.

Hierarchical Solution

A two-stage filter reduces the search space:

Stage 1 — Feature-space KNN filter:

Extract a compact feature vector from each light curve:

$$\phi(X) = [\mu,\ \text{med},\ \min,\ \max,\ \sigma,\ \text{skew},\ \kappa,\ n_{\text{peaks}},\ n_{\text{troughs}},\ f_1,\ f_2,\ f_3]$$

where f_1, f_2, f_3 are the dominant FFT frequency bins. The DFT of the light curve is:

$$\hat{X}[k] = \sum_{n=0}^{N-1} x_n \cdot e^{-2\pi i k n / N}$$

The DC component is zeroed out ($\hat{X}[0] = 0$) and the top-3 dominant bins are:

The DC component is zeroed out ($\hat{X}[0] = 0$) and the top-3 dominant bins are:

$$f_j = \arg\operatorname{sort}_k \left(|\hat{X}[k]| \right)[-j], \quad j = 1, 2, 3$$

Nearest neighbours are found in feature space using Manhattan (L1) distance:

$$d_{\ell_1}(\phi(X), \phi(Z)) = \sum_i |\phi(X)_i - \phi(Z)_i|$$

Stage 2 — DTW re-ranking:

Apply DTW only within the k candidate neighbours returned by Stage 1. Assign the label of the DTW-closest match. This reduces per-test-example complexity from $\mathcal{O}(m \cdot n \cdot p)$ to $\mathcal{O}(m \cdot n \cdot k)$, where $k \ll p$.

Results

Model	Class	Precision	Recall	F1
Baseline KNN (k=20, Manhattan)	Non-host	0.96	0.97	0.97
	Exoplanet	0.75	0.65	0.70
	Overall accuracy			**0.94**
Hierarchical KNN+DTW (k=10)	Non-host	0.97	0.98	0.98
	Exoplanet	0.82	0.75	0.78
	Overall accuracy			**0.95**

DTW post-filtering improved exoplanet precision by +7pp and recall by +10pp without sacrificing non-host classification.

---

3. 2D CNN on Phase-Folded Light Curve Images

Phase Folding

A light curve $F(t)$ is phase-folded at period P by mapping each timestamp t to a phase $\phi \in [0, 1]$:

$$\phi(t) = \frac{t \bmod P}{P}$$

This stacks all transit events on top of each other, producing a folded curve with a clearly

visible dip. Each folded light curve is rendered as a 128×128 grayscale image and normalised:

$$\tilde{I}(x, y) = \frac{I(x, y)}{255}$$

Convolutional Layers

A 2D convolutional layer applies learned filters K over the input image I:

$$S(i, j) = (I * K)(i, j) = \sum_{m} \sum_{n} I(i + m,\ j + n) \cdot K(m, n)$$

Each filter learns to detect a local spatial pattern — e.g. the characteristic symmetric dip shape of a planetary transit. Successive layers with increasing filter counts (32 → 64 → 128) learn increasingly abstract representations.

Activation and Pooling

ReLU activation introduces non-linearity after each convolution:

$$\text{ReLU}(x) = \max(0, x)$$

Max pooling reduces spatial dimensionality and provides local translation invariance:

$$P(i, j) = \max_{(m,n) \in \text{window}} S(i + m,\ j + n)$$

Architecture

```
Input: 128×128×1 grayscale image
→ Conv2D(32, 3×3) + ReLU + MaxPool(2×2)
→ Conv2D(64, 3×3) + ReLU + MaxPool(2×2)
→ Conv2D(128, 3×3) + ReLU + MaxPool(2×2)
→ Flatten
→ Dense(64, ReLU)
→ Dropout(0.4)
→ Dense(1, Sigmoid)
```

Loss function — binary crossentropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i) \right]$$

Optimised with Adam. Early stopping monitors validation loss with patience=3.

Class Imbalance

Confirmed exoplanet hosts are a small minority of all Kepler targets. Three measures were applied:

- Positive class (exoplanet hosts) oversampled to balance training distribution
- Random temporal shifts applied as data augmentation to improve generalisation
- Classification threshold tuned to 0.67 (rather than default 0.50) based on the precision-recall tradeoff — maximising the F1-equivalent operating point on the ROC curve (AUC: 0.97)

Results

```
Test accuracy:     99.20%
AUC score:         0.97
Optimal threshold: 0.67
Training images:   5,657 (3 corrupted files skipped)
```

Remaining misclassifications arise from:

- **Eclipsing binaries**: produce periodic transit-like dips photometrically indistinguishable from planetary transits
- **Residual PRF systematics**: instrumental effects not fully removed by detrending, implicitly learned but occasionally misleading

---

Repository Structure

```
├── DTW_Kepler.ipynb          # Hierarchical KNN+DTW classifier
├── cnn_2d_model.ipynb        # 2D CNN training and evaluation
├── cnn_2d_image.ipynb        # Phase-folding and image generation
├── END_TERM_REPORT.pdf       # Full mathematical report with derivations
├── requirements.txt
└── README.md
```

Installation

bash

```
git clone https://github.com/piyarshah/kepler-exoplanet-detection
cd kepler-exoplanet-detection
pip install -r requirements.txt
```

Data

Light curve data sourced from the Kepler mission via the MAST archive. Additional labels from the NASA Exoplanet Archive.

The CNN training set consists of 5,660 phase-folded light curve images labelled by KOI disposition (0 = non-host, 1 = confirmed exoplanet host).

References

- Shallue & Vanderburg (2018) — AstroNet: dual-branch CNN for Kepler exoplanet detection
- Tiensuu et al. (2019) — Image encoding of light curves for CNN classification
- Malik et al. (2020) — Cross-mission pipeline; 98% accuracy on TESS, AUC 0.948 on Kepler
- Priyadarshini & Puri (2021) — Stacked CNN ensemble, 99.62% accuracy on Kepler
- Feinstein et al. (2019) — Lightkurve library
- Bishop (2006) — Pattern Recognition and Machine Learning

Full bibliography in `END_TERM_REPORT.pdf`.